La Componente Cloud di un Sistema Distribuito Edge–Cloud per l'Analisi di Macchie di Sangue

Emanuele Ugolotti
Corso: Cloud and Edge Computing
[Unimore]

26 settembre 2026

Sommario

Questo report si concentra sulla componente Cloud di un sistema distribuito per la classificazione di macchie di sangue, le cui componenti edge e mobile sono descritte in un report separato. Vengono trattate le scelte progettuali alla base dell'architettura del backend, il flusso dei dati attraverso il Cloud, dove risiede il modello di classificazione e perché, gli esperimenti di robustezza adversarial condotti su di esso, e le differenze tra il deployment con Docker Compose usato in sviluppo e un equivalente deployment con Docker Swarm realizzato per dimostrare la scalabilità orizzontale.

1 Ruolo del livello Cloud

In questo sistema, un gateway IoT inoltra le acquisizioni di immagini (sia riferimenti a un dataset condiviso, sia fotografie scattate sul campo) verso il Cloud quando è raggiungibile, ricorrendo altrimenti all'inferenza in locale su un'applicazione mobile di supporto. Il Cloud svolge quindi quattro ruoli: è il bersaglio di inferenza *primario* (più potenza di calcolo rispetto al telefono), è l'*archivio di riferimento* per ogni acquisizione e il suo risultato, è l'ambiente in cui vengono eseguiti gli *esperimenti di robustezza adversarial* contro il modello, ed espone infine una *superficie di visualizzazione* leggera (sia verso l'app mobile sia tramite una dashboard web integrata) per ispezionare i risultati senza bisogno di accesso diretto al database.

2 Architettura e scelte progettuali

Il backend è un piccolo insieme di container Docker: `api` (FastAPI, espone tutti gli endpoint HTTP), `db` (PostgreSQL), e `robustness` (TensorFlow, ospita il classificatore VGG16 e gli attacchi adversarial). Due scelte meritano di essere evidenziate:

Un solo servizio di inferenza, riusato per due scopi. Invece di far girare un servizio leggero separato solo per la classificazione e un altro per gli attacchi adversarial, entrambi i bisogni sono serviti dallo stesso container `robustness` e dalla stessa istanza del modello già caricata in memoria. Caricare un modello VGG16 due volte raddoppierebbe l'uso di memoria senza alcun beneficio, dato che il codice degli attacchi ha bisogno esattamente della stessa capacità di calcolo (forward pass) richiesta dalla semplice inferenza.

L'ingest è disaccoppiato dalla classificazione. L'endpoint di ingest salva sempre prima l'acquisizione, e solo dopo tenta la classificazione; un servizio di inferenza temporaneamente irraggiungibile o lento non può mai far perdere un'acquisizione, al massimo la lascia non classificata fino al prossimo tentativo. Questo privilegia la persistenza dei dati rispetto all'immediatezza, scelta fatta per un sistema la cui fonte a monte (un microcontrollore con vincoli di batteria e di radio) non può facilmente essere invitata a rimandare una trasmissione fallita.

Un'unica tabella `acquisitions` memorizza tutti e tre i tipi di evento (riferimenti al dataset, foto catturate, e tentativi di scatto falliti, distinti da una colonna di modalità/stato) invece di tre tabelle normalizzate separate.

3 Flusso dei dati

Un'acquisizione in arrivo viene validata, salvata (incluso il file immagine, per le foto catturate), e — se il servizio del modello è raggiungibile — classificata immediatamente; l'etichetta predetta e l'intera distribuzione di proba-

bilità per classe vengono salvate insieme ad essa. I tentativi di scatto falliti (segnalati dal gateway quando una trasmissione non può essere completata o verificata) vengono salvati allo stesso modo, senza immagine, puramente come traccia che uno scatto va rifatto. Sia i risultati classificati sia i fallimenti possono essere interrogati tramite endpoint dedicati, consumati sia dall'applicazione mobile sia da una dashboard HTML minimale integrata (HTML e JavaScript semplice, nessun framework) che elenca i risultati con miniature e il dettaglio completo delle probabilità, e permette di ingrandire qualunque immagine — utile per dimostrare il lato Cloud del sistema senza dover avere il telefono a portata di mano.

4 Dove risiede il modello

Il classificatore VGG16 addestrato viene caricato una sola volta, all'avvio del container, dentro il servizio `robustness`, e tenuto in memoria per tutta la durata del container; ogni classificazione e ogni chiamata di attacco adversarial riusano questa stessa istanza, invece di ricaricare i pesi a ogni richiesta. Una copia *separata* degli stessi pesi addestrati, convertita in TensorFlow Lite, è incorporata direttamente nell'applicazione Android. Questa duplicazione è intenzionale: permette al telefono di classificare in autonomia quando il Cloud non è raggiungibile, invece di essere un client sottile inutile senza connettività — a supporto diretto della strategia di offloading descritta nel report separato.

5 Esperimenti di robustezza adversarial

Sono implementati ed esposti dal Cloud due attacchi adversarial black-box: il *Boundary Attack* e *SimBA*. Entrambi operano senza bisogno di accedere ai gradienti del modello, perturbando un'immagine in ingresso in modo iterativo, interrogando solo le predizioni in uscita.

Un risultato metodologicamente rilevante, è che entrambi gli attacchi sono riusciti a cambiare la classe predetta dal modello VGG16 sul dataset vero di macchie di sangue usato in questo progetto, in tutti e tre i tentativi effettuati. Va però segnalato un limite qualitativo importante: le perturbazioni necessarie per ottenere questo cambiamento sono risultate visibili a occhio nudo, non impercettibili come normalmente ci si aspetterebbe da un esempio adversarial "riuscito" in senso stretto — un risultato parziale, non un fallimento, ma da interpretare con questa cautela.

Una spiegazione plausibile del perché servano perturbazioni così ampie è che la pipeline di preelaborazione delle immagini applicata prima della classificazione (in particolare il ridimensionamento) comporti anche una perdita di accuratezza generale del modello, osservata durante gli stessi esperimenti. Durante un lavoro di tirocinio precedente sullo stesso dominio, usando questi stessi due attacchi (gli unici, tra quelli provati, ad aver prodotto risultati positivi, per quanto limitati), le previsioni del modello non avevano mai raggiunto lo stesso livello di certezza quasi assoluta osservato in questo progetto specifico. La causa di questa differenza a mio parere resta il ridimensionamento delle immagini e quindi la perdita di informazioni.

6 Docker Compose contro Docker Swarm

Il sistema viene sviluppato e normalmente eseguito con Docker Compose, che costruisce le immagini dei container direttamente dal codice sorgente su richiesta e supporta un ordine di avvio consapevole dello stato di salute (un servizio dipendente può essere fatto attendere finché un altro non si segnala come pronto). Come esercizio di deployment, è stato realizzato e testato un equivalente file di stack per *Docker Swarm* (su uno swarm a un solo nodo, senza bisogno di macchine fisiche aggiuntive). I due differiscono per diversi aspetti rilevanti, riassunti nella Tabella 1.

In questo progetto, tutti e quattro i servizi sono stati distribuiti con successo sotto Swarm, raggiungendo il numero di repliche atteso e rispondendo correttamente. L'assenza di un ordine di avvio consapevole dello stato di salute è stata osservata direttamente (il servizio `robustness` ha momentaneamente segnalato zero repliche disponibili durante il caricamento del modello), a conferma che questa distinzione conta nella pratica, non solo sulla carta.

	Docker Compose	**Docker Swarm**
Origine immagine	Costruita al volo da un `Dockerfile`	Deve essere già costruita e taggata in anticipo
Ordine di avvio	Può attendere l'healthcheck di una dipendenza	Non è consapevole dello stato di salute; un servizio dipendente può partire prima che la sua dipendenza sia pronta
Scalabilità	Una sola istanza per servizio, un solo host	`deploy.replicas` — il punto naturale per far girare più istanze, anche su più nodi fisici collegati allo stesso swarm
Uso previsto	Sviluppo locale	Orchestrazione in stile produzione

Tabella 1: Differenze principali tra il deployment con Compose e quello con Swarm dello stesso sistema.

7 Limiti noti

Due limiti meritano di essere segnalati esplicitamente. Primo, l'autenticazione (basata su chiave API) è implementata e attivabile tramite configurazione, ma resta disattivata nella configurazione dimostrativa per semplicità. Secondo, come qualunque classificatore a insieme chiuso, il modello restituisce sempre una distribuzione di probabilità completa sulle classi note anche per immagini estranee al proprio dominio di addestramento, dato che l'ultimo strato softmax non ha alcuna nozione di "nessuna delle precedenti" — un limite generale della classificazione supervisionata, non specifico di questo deployment.

8 Conclusione

La componente Cloud di questo sistema è stata progettata attorno alla resilienza (memorizzazione persistente disaccoppiata dall'inferenza), all'efficienza delle risorse (un'unica istanza di modello condivisa per classificazione e test di robustezza), e a un percorso dimostrabile dallo sviluppo (Compose) verso un deployment in stile produzione (Swarm). I risultati sulla robustezza adversarial, pur non ancora positivi sul dataset proprio del progetto, sono riportati come un'osservazione empirica, piuttosto che presentati in modo da apparire più conclusivi di quanto le prove raccolte permettano.